Customer Segmentation using RFM Analysis

1. Abstract

This project focuses on analyzing customer transaction data using RFM (Recency, Frequency, Monetary) analysis to segment customers based on their purchasing behavior. The objective is to identify high-value, loyal, and at-risk customers to support better business decision-making. The project involves data cleaning, feature engineering, and segmentation using Python, followed by the development of an interactive dashboard in Power BI. The results help businesses understand customer patterns and improve marketing strategies.

2. Problem Statement

Businesses often struggle to identify valuable customers and optimize marketing strategies. Without proper segmentation, companies fail to retain high-value customers and cannot target at-risk customers effectively. This project aims to solve this problem by analyzing customer behavior and segmenting them into meaningful groups using RFM analysis.

3. Objectives

- To analyze customer purchasing behavior
- To perform RFM (Recency, Frequency, Monetary) analysis
- To segment customers into different categories
- To identify high-value and at-risk customers
- To build an interactive dashboard for insights

4. Dataset Description

The dataset contains retail transaction data with the following columns:

	A	B	C	D	E	F	G	H	I
	Transaction ID	Date	Customer	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
2	1	24-11-2023	CUST001	Male	34	Beauty	3	50	150
3	2	27-02-2023	CUST002	Female	26	Clothing	2	500	1000
4	3	13-01-2023	CUST003	Male	50	Electronics	1	30	30
5	4	21-05-2023	CUST004	Male	37	Clothing	1	500	500
6	5	06-05-2023	CUST005	Male	30	Beauty	2	50	100
7	6	25-04-2023	CUST006	Female	45	Beauty	1	30	30
8	7	13-03-2023	CUST007	Male	46	Clothing	2	25	50
9	8	22-02-2023	CUST008	Male	30	Electronics	4	25	100
10	9	13-12-2023	CUST009	Male	63	Electronics	2	300	600
11	10	07-10-2023	CUST010	Female	52	Clothing	4	50	200

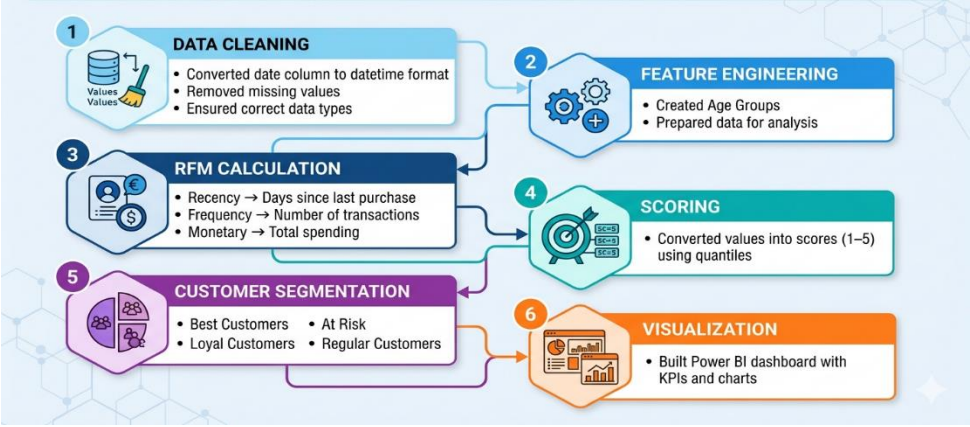
- **Transaction ID** – Unique identifier for each transaction
- **Date** – Date of transaction
- **Customer ID** – Unique customer identifier
- **Gender** – Male/Female
- **Age** – Age of customer
- **Product Category** – Category of product purchased
- **Quantity** – Number of items purchased
- **Price per Unit** – Price of one unit
- **Total Amount** – Total transaction value

5. Tools & Technologies

- **Python (Pandas, NumPy, Matplotlib)** – Data processing & analysis
- **SQL (conceptual use)** – Data querying logic
- **Power BI** – Dashboard creation
- **Excel** – Initial data exploration

6. Methodology

The project follows these steps:



1. Data Cleaning

- Converted date column to datetime format
- Removed missing values
- Ensured correct data types

2. Feature Engineering

- Created Age Groups
- Prepared data for analysis

3. RFM Calculation

- Recency → Days since last purchase
- Frequency → Number of transactions
- Monetary → Total spending

4. Scoring

- Converted values into scores (1–5) using quantiles

5. Customer Segmentation

- Categorized customers into:
    - Best Customers
    - Loyal Customers
    - At Risk
    - Regular Customers

6. Visualization

- Built Power BI dashboard with KPIs and charts

7. RFM Explanation

RFM Analysis is a technique used to segment customers based on their purchasing behavior using three key metrics: Recency, Frequency, and Monetary.

- **Recency (R):** Number of days since the customer's last purchase. Customers with low recency are more active.
- **Frequency (F):** Total number of transactions made by a customer. Higher frequency indicates more loyal customers.
- **Monetary (M):** Total amount spent by the customer. Higher monetary value indicates high-value customers.

Customers with **low recency, high frequency, and high monetary value** are considered the most valuable, while customers with **high recency and low frequency** are considered at risk.

Customers with:

- Low Recency = active
- High Frequency = loyal
- High Monetary = valuable

8.Implementation

The implementation was done using Python in Jupyter.

Step 1 :- Setup and Data Loading

```
[6]: # downloads dependies
     import pandas as pd
     import warnings
     warnings.filterwarnings('ignore')

[7]: # importig dataset
     df = pd.read_csv("Downloads/retail_sales_dataset.csv")

[8]: df.head()
```

[8]:

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
0	1	24-11-2023	CUST001	Male	34	Beauty	3	50	150
1	2	27-02-2023	CUST002	Female	26	Clothing	2	500	1000
2	3	13-01-2023	CUST003	Male	50	Electronics	1	30	30
3	4	21-05-2023	CUST004	Male	37	Clothing	1	500	500
4	5	06-05-2023	CUST005	Male	30	Beauty	2	50	100

- **import pandas as pd**: Imports the Pandas library, used for data manipulation and analysis.
- **import warnings; warnings.filterwarnings('ignore')**: Suppresses warning messages to keep the notebook output clean.
- **df = pd.read_csv("Downloads/retail_sales_dataset.csv")**: Loads the sales data from a CSV file into a DataFrame named df.
- **df.head()**: Displays the first five rows of the dataset to verify it loaded correctly.

Step 2 :- Data Cleaning and Preprocessing

- **df.isnull().sum()**: Checks for missing values in each column.
- **df.info()**: Shows the data types and non-null counts for the dataset.

```
[10]: # checking missing data in dataset
      df.isnull().sum()
```

```
[10]: Transaction ID      0
      Date                0
      Customer ID         0
      Gender              0
      Age                 0
      Product Category    0
      Quantity            0
      Price per Unit      0
      Total Amount        0
      dtype: int64
```

```
[11]: # datatype checking
      df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    1000 non-null   int64
 1   Date              1000 non-null   datetime64[ns]
 2   Customer ID       1000 non-null   object
 3   Gender            1000 non-null   object
 4   Age               1000 non-null   int64
 5   Product Category  1000 non-null   object
 6   Quantity          1000 non-null   int64
 7   Price per Unit    1000 non-null   int64
 8   Total Amount      1000 non-null   int64
dtypes: datetime64[ns](1), int64(5), object(3)
memory usage: 70.4+ KB
```


```
 6   Quantity          1000 non-null   int64
 7   Price per Unit    1000 non-null   int64
 8   Total Amount      1000 non-null   int64
dtypes: datetime64[ns](1), int64(5), object(3)
memory usage: 70.4+ KB
```


```
[32]: # fixing date column data type
      df['Date'] = pd.to_datetime(df['Date'])
```

```
[12]: # converting total amount dataset into float data set
      df['Total Amount'] = df['Total Amount'].astype(float)
```

- **df['Date'] = pd.to_datetime(df['Date'])**: Converts the 'Date' column from a string (object) to a datetime format so calculations can be performed on it.
- **df['Total Amount'] = df['Total Amount'].astype(float)**: Ensures the 'Total Amount' column is treated as a floating-point number for financial calculations.

Step 3 :- Creating RFM Metrics

RFM stands for:

- **Recency**: How many days since the customer's last purchase?

Create RFM Metrics

3]:
```python
# calculating how many days have passed since a customer last bought something
import datetime
reference_date = df['Date'].max() + datetime.timedelta(days=1)
```

4]:
```python
# Recency, Frequency, Monetary
rfm = df.groupby('Customer ID').agg({
    'Date': lambda x: (reference_date - x.max()).days,
    'Transaction ID': 'count',
    'Total Amount': 'sum'
})

rfm.columns = ['Recency', 'Frequency', 'Monetary']
rfm.reset_index(inplace=True)

rfm.head()
```

4]:

	Customer ID	Recency	Frequency	Monetary
0	CUST001	39	1	150.0
1	CUST002	309	1	1000.0
2	CUST003	354	1	30.0
3	CUST004	226	1	500.0
4	CUST005	241	1	100.0

- **Frequency**: How many transactions did they make?
- **Monetary**: How much total money did they spend?
- **import datetime**: Imports the datetime module.
- **reference_date = df['Date'].max() + datetime.timedelta(days=1)**: Sets a reference point (usually the day after the most recent purchase in the data) to calculate how "recent" a customer's last visit was.
- **rfm = df.groupby('Customer ID').agg({...})**: Groups the data by each unique customer and calculates the three metrics:
    - **Recency**: (reference_date - x.max()).days (Days since last purchase).
    - **Frequency**: 'Transaction ID': 'count' (Total number of orders).
    - **Monetary**: 'Total Amount': 'sum' (Total spend).
- **rfm.columns = ['Recency', 'Frequency', 'Monetary']**: Renames the resulting columns for clarity.

- **rfm.reset_index(inplace=True)**: Converts 'Customer ID' back from an index to a regular column.

Step 4 :- Scoring and Segmentation

```
[15]: # Create 5 equal-sized buckets for Recency.
      # Note: Higher score (5) means fewer days since last visit (Better).
      rfm['R_score'] = pd.qcut(rfm['Recency'], 5, labels=[5,4,3,2,1])

[16]: # Create 5 equal-sized buckets for frequency.
      # Note: Higher score (5) means fewer days since last visit (Better).
      rfm['F_score'] = rfm['Frequency'].rank(method='first')
      rfm['F_score'] = pd.qcut(rfm['F_score'], 5, labels=[1,2,3,4,5])

[17]: # Create 5 equal-sized buckets for amount of money did each coustomer pay .
      # Note: Higher score (5) means fewer days since last visit (Better).
      rfm['M_score'] = pd.qcut(rfm['Monetary'], 5, labels=[1,2,3,4,5])

[19]: rfm['RFM_Score'] = (
          rfm['R_score'].astype(str) +
          rfm['F_score'].astype(str) +
          rfm['M_score'].astype(str)
      )

      rfm.head()
```

[19]:

	Customer ID	Recency	Frequency	Monetary	R_score	F_score	M_score	RFM_Score
0	CUST001	39	1	150.0	5	1	3	513
1	CUST002	309	1	1000.0	1	1	4	114
2	CUST003	354	1	30.0	1	1	1	111
3	CUST004	226	1	500.0	2	1	4	214
4	CUST005	241	1	100.0	2	1	2	212

The code assigns a score from 1 to 5 for each metric (Recency, Frequency, Monetary).

- **pd.qcut(rfm['Recency'], 5, labels=[5,4,3,2,1])**: Divides customers into 5 equal groups based on Recency. Note that **5 is the best score** (most recent).
- **rfm['F_score'] = ... and rfm['M_score'] = ...**: Similarly assigns scores for Frequency and Monetary (where 5 is the highest/best).
- **rfm['RFM_Score'] = rfm['R_score'].astype(str) + ...**: Concatenates the three scores into a single string (e.g., a "555" score is a perfect customer).

Step 5 :-Categorizing Customers

A custom function segment(row) is used to label customers based on their scores:

- **Best Customers**: Score of '555'.
- **Loyal Customers**: Frequency score of 4 or higher.

```python
# creating customer Category
def segment(row):
    if row['RFM_Score'] == '555':
        return 'Best Customers'
    elif int(row['F_score']) >= 4:
        return 'Loyal Customers'
    elif int(row['R_score']) <= 2:
        return 'At Risk'
    else:
        return 'Regular Customers'

rfm['Segment'] = rfm.apply(segment, axis=1)

rfm.head()
```

	Customer ID	Recency	Frequency	Monetary	R_score	F_score	M_score	RFM_Score	Segment
0	CUST001	39	1	150.0	5	1	3	513	Regular Customers
1	CUST002	309	1	1000.0	1	1	4	114	At Risk
2	CUST003	354	1	30.0	1	1	1	111	At Risk
3	CUST004	226	1	500.0	2	1	4	214	At Risk
4	CUST005	241	1	100.0	2	1	2	212	At Risk

- **At Risk**: Recency score of 2 or lower (haven't visited in a long time).
- **Regular Customers**: Everyone else.
- **rfm['Segment'] = rfm.apply(segment, axis=1)**: Applies this logic to every row in the RFM table.

Step 6 :- Exporting Results

```python
[30]: # exporting output file
      rfm.to_csv("rfm_output.csv", index=False)

[31]: from IPython.display import FileLink

      # This creates a blue clickable link in your notebook
      FileLink("rfm_output.csv")

[31]: rfm_output.csv
```

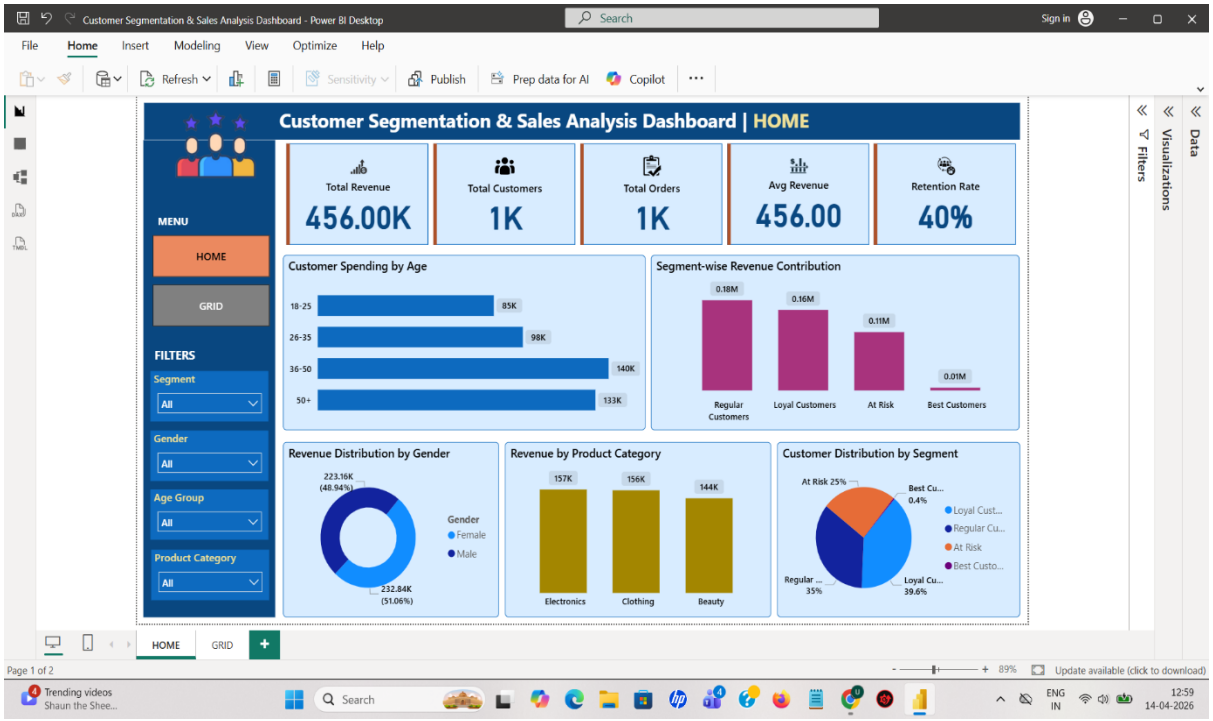
- **rfm.to_csv("rfm_output.csv", index=False)**: Saves the final segmented customer list to a new CSV file.
- **FileLink("rfm_output.csv")**: Generates a clickable link in the notebook to download the resulting file.

9. Dashboard Overview

The dashboard was built using Power BI to visualize insights.

Key Components:

- **KPI Cards:**
  - Total Revenue
  - Total Customers
  - Total Orders
  - Average Revenue
  - Retention Rate
- **Charts:**
  - Customer Distribution by Segment
  - Revenue Contribution by Segment
  - Revenue by Gender
  - Spending by Age Group
  - Revenue by Product Category
- **Filters:**
  - Segment
  - Gender
  - Age Group
  - Product Category



10. Key Insights

- A small group of customers contributes the majority of revenue
- Loyal customers generate consistent revenue
- At-risk customers require retention strategies
- Age group 26–35 contributes the highest spending
- Product categories like Electronics and Clothing perform best

11. Conclusion

RFM analysis is an effective technique for customer segmentation. This project demonstrates how businesses can use data to identify valuable customers, improve retention strategies, and increase revenue. The dashboard provides actionable insights for decision-making.